PROGRESS SEMINAR BRIEF

VanniKawachh

Acoustic distress detection network with autonomous drone response
Everything you need to answer the panel - nothing you don't

1 The pitch - five sentences to know cold

1. Solar-powered ESP32-S3 nodes on street poles listen continuously and run a tiny neural network on-device to spot a scream or a call for help. That is Stage 1.
2. A hit sends a 25-byte encrypted, authenticated alert over LoRa radio - no internet, 5 to 10 km range - plus a 4-second audio clip over WiFi.
3. A Raspberry Pi 5 hub re-checks that clip with a much larger pretrained audio model (Stage 2), combines it with motion, darkness and time-of-day evidence, and only then decides.
4. If two independent thresholds are both crossed, the hub sends the node's surveyed coordinates to the drone stack, which flies a fully autonomous mission: arm, take off, navigate, hover and record evidence, drop a first-aid kit, return to launch. Zero manual piloting.
5. The novelty is not 'send a drone to a GPS point' - that already exists. It is the two-stage verification before an aircraft is ever committed, plus a flight stack where every command is confirmed by the autopilot before the mission proceeds.

Repository	github.com/SV-1411/drone
Languages	Python 3.11 (hub, flight stack, ML), C/C++ Arduino (ESP32 firmware), React (dashboard)
Size	~10,700 lines, 60 source files
Test status	80 unit tests pass (re-run and verified); end-to-end autonomous flight validated in ArduPilot SITL
Stage reached	Phase 0 complete - full chain works in simulation. Hardware bring-up is Phase 1.

2 Architecture - draw this if asked

DATA FLOW

```
SENSING NODE     ESP32-S3 + INMP441 mic + PIR + LDR + SX1278 LoRa
      |   25-byte encrypted alert   (LoRa 433 MHz)      [no internet needed]
      |   4 s audio clip            (WiFi)
      v
HUB              Raspberry Pi 5  +  gateway ESP32 (LoRa receiver on USB)
      |   authenticate -> verify audio -> fuse evidence -> decide
      |   POST /trigger {lat, lon, priority}
      v
TRIGGER API      FastAPI + priority queue + SQLite
      v
FLIGHT CORE      13-state machine + failsafe monitor + MAVLink client
      |   MAVLink over TCP (simulator) or UART (real Pixhawk)
      v
AIRCRAFT         Pixhawk running ArduPilot + servo kit release + Pi camera

DASHBOARD        React + Leaflet map  <--HTTP/WebSocket-->  TRIGGER API
```

Why two stages? (the most common question)

The big audio model is about 80 MB of weights. An ESP32-S3 has about 512 KB of memory. There is no single model that fits both the pole and the accuracy requirement. So Stage 1 is a deliberately tiny, high-recall detector on the node that only decides 'worth a look', and Stage 2 is the heavy, high-precision model at the hub that decides 'actually distress'. This is a cascade: cheap filter first, expensive confirmation second.

- Bandwidth - under 0.2% of audio is ever transmitted, so a solar node on a LoRa link is enough.
- Privacy - no continuous audio leaves the pole, only 4 seconds around a detection, and it goes to a local hub, not the cloud.
- False alarms - a needless drone launch is expensive and erodes public trust, so precision is bought back at Stage 2 where compute is cheap.
- Power - inference twice a second on 2-second windows fits a solar and 18650 budget; continuous streaming would not.

3 Which code runs on which hardware

THE TRAP QUESTION

"What code do you flash onto the Pixhawk?" - None. The Pixhawk runs stock ArduPilot firmware, flashed once with Mission Planner. Our Python code is an external client that commands it over MAVLink. Only the two ESP32 boards get code we wrote and burned; the Pi and the laptop have software installed, not burned.

Hardware	What we put on it	What that code does
ESP32-S3 (sensing node)	firmware/node/ - node.ino, stage1.cpp, stage1_nn.h (the trained weights). Flashed from Arduino IDE over USB.	Captures audio at 16 kHz, keeps a 4-second circular buffer, extracts features and classifies every half second, reads the motion and light sensors, encrypts and signs the alert, transmits over LoRa, uploads the clip over WiFi
ESP32 (LoRa gateway)	firmware/gateway/gateway.ino. Flashed from Arduino IDE.	A deliberately dumb bridge: receives LoRa frames and prints them as hex on USB serial. Holds no keys and does no verification, so stealing it gains an attacker nothing
Raspberry Pi 5 (hub)	The hub/ Python package. Installed with pip, run as python -m hub.main	Reads the gateway, authenticates and decrypts packets, rejects replays, looks up where the node is, runs Stage-2 verification on the clip, fuses the evidence, applies the two thresholds, calls the drone API, serves the dashboard
Pi Zero 2 W (on the drone)	flight_core/ + trigger_api/. Installed with pip, run under systemd.	Receives the trigger, queues it by priority, runs the flight state machine, sends MAVLink commands to the Pixhawk, runs the failsafe monitor once a second, records evidence video, fires the payload servo, streams telemetry
Pixhawk 2.4.8	NOTHING OF OURS. Stock ArduPilot Copter plus a parameter set.	Runs the actual flight control loops, sensor fusion, its own hardware failsafes and geofence. We only send it MAVLink messages and read its telemetry back
Laptop / PC	The whole repo: the flight simulator, the dashboard, the training scripts, the test suite	Simulates the aircraft so the entire stack can fly with no hardware; trains and exports the Stage-1 model; runs the 80 tests
A phone (demo only)	Nothing installed - it opens a web page the hub serves	Lets you demo the whole detection-to-dispatch chain with no hardware: the phone records real audio, the hub runs the real Stage 1 and Stage 2, and a drone animates on the map

Node wiring - memorise this

Peripheral	Signal	ESP32-S3 GPIO
INMP441 mic (I2S)	WS (word select)	GPIO 4
INMP441 mic (I2S)	SCK (bit clock)	GPIO 5

Peripheral	Signal	ESP32-S3 GPIO
INMP441 mic (I2S)	SD (data in)	GPIO 6
INMP441 mic	L/R to GND (selects left channel), VDD to 3V3	-
SX1278 LoRa (SPI)	SCK / MISO / MOSI	GPIO 12 / 13 / 11
SX1278 LoRa	NSS (chip select) / RST / DIO0	GPIO 10 / 9 / 8
HC-SR501 PIR	OUT (digital)	GPIO 7
LDR + 10k divider	analog in	GPIO 1

Gateway ESP32 (different board, different pins): NSS 5, SCK 18, MOSI 23, MISO 19, RST 14, DIO0 2. Both radios run at 433 MHz with spreading factor 9 - node and gateway must match or nothing is received.

> **KNOW THIS BEFORE THEY FIND IT**
> docs/HARDWARE_INTEGRATION.md lists the mic as WS = GPIO 5 and SCK = GPIO 4, which is swapped relative to the firmware (node.ino sets WS = 4, SCK = 5). The same paragraph says '1 s windows' where the code uses 2 s. Both are documentation bugs, not code bugs. The firmware is the source of truth - say so plainly if it comes up.

Hardware rules never to get wrong

- Never transmit on the LoRa module without the antenna attached - the power amplifier will burn out.
- The Pixhawk servo rail is not powered internally - the kit-release servo needs a separate 5 V supply.
- The environment variable SITL_MODE=1 relaxes the autopilot's pre-arm checks. It is for the simulator only and must stay unset on real hardware - the code refuses to touch any parameter unless it is set.
- Every real flight needs an RC transmitter with a mode switch, even though the mission is autonomous. That is the human kill path.

4 How one scream becomes a flight

Eight steps. If you can narrate these, you can answer almost any 'how does it work' question.

1. **Listen and classify (on the node)**
   Audio is captured at 16 kHz into a 4-second circular buffer. Every half second the newest 2 seconds are turned into features and classified into one of four classes: background, scream, cry, or help. Because the buffer is circular, the clip that later gets uploaded includes the audio from BEFORE the trigger - which is the part Stage 2 needs most.
2. **Decide to alert (on the node)**
   Fire only if the class is not background, the confidence is at least 0.60, and at least 15 seconds have passed since the last alert. That last condition is a refractory period - one incident should produce one alert, not thirty.
3. **Add context and seal (on the node)**
   Read the motion sensor and the light level. Then build a 25-byte packet: encrypt the payload with AES-128 and append a signature computed with a key unique to that node. Transmit over LoRa, then upload the 4-second clip over WiFi.
4. **Bridge (gateway ESP32)**
   The gateway receives the radio frame and prints it as hex on USB serial. It never decrypts anything.
5. **Authenticate (hub)**
   The hub checks the length and format, recomputes the signature and compares it in constant time, and rejects any message counter that is not higher than the last one seen from that node. Only then does it decrypt. Any failure means the packet is dropped and no drone moves.
6. **Locate and verify (hub)**
   The packet carries only a node ID, so a registry supplies the surveyed coordinates - which is why no node needs its own GPS module. The hub then waits up to 8 seconds for the audio clip and scores it with the large Stage-2 model. If the clip never arrives, the score degrades to 60% of the node's own confidence: the system keeps working on radio alone, just more conservatively.

7. **Fuse and decide (hub)**
   The audio score is combined with the node's confidence, whether motion was detected, how dark it is, and whether it is night, producing a single severity between 0 and 1. Two independent gates must both pass before an aircraft is committed. Otherwise the incident is logged with no dispatch.

8. **Fly (drone stack)**
   The API validates the coordinates and rejects anything outside the geofence, then queues the mission by priority. The state machine runs it: connect, wait for GPS lock, confirm GUIDED mode, confirm armed, climb to altitude, navigate to the point routing around any configured no-fly zones, hover while recording evidence, descend to 3 m and release the first-aid kit, then return to launch and confirm landed and disarmed.

THE ANSWER TO 'WHY IS THIS SAFE?'

Three mechanisms. (1) Verified commands: no flight-mode change is ever assumed - it is re-sent every 700 milliseconds until the autopilot's own heartbeat reports the new mode back. (2) Landing interlock: every abort blocks until the aircraft has landed AND disarmed, so the queue can never start a flight against an airborne vehicle. (3) Failsafe priority: LAND outranks RETURN-TO-LAUNCH and is never downgraded, and GPS loss must persist for three consecutive seconds before it acts.

5 The algorithms, one line each

What each one does and why it is there. The four formulas worth memorising are boxed.

Algorithm	What it does	Where it lives
MFCC feature extraction	Turns 32,000 raw audio samples into 123 x 13 numbers describing the shape of the sound spectrum the way human hearing perceives it. Written by hand in both Python and C so the features on the device exactly match the features the model trained on.	ml/mfcc.py and firmware/node/stage1.cpp
Stage-1 classifier	A tiny neural network: 26 inputs, one hidden layer of 24, four outputs. 748 parameters, about 3 KB, needs no machine-learning library on the device at all. A larger convolutional version exists in the code for when more data is available.	firmware/node/stage1_nn.h (weights), stage1.cpp (maths)
FFT	Converts each 32-millisecond audio frame from time into frequency. Hand-written on the device because it runs 123 times per window and must be fast.	firmware/node/stage1.cpp
Stage-2 verification (PANNs)	A large pretrained model (CNN14, trained on Google's AudioSet: 2 million clips, 527 sound classes) scores the clip by adding up the probabilities of the distress-related classes - screaming, shouting, yelling, crying.	hub/verifier.py
Stage-2 fallback	A simple hand-designed scorer using loudness, how high in the spectrum the sound sits, and how bursty it is. It exists so the whole chain can be demonstrated on any laptop without a 2 GB install. It is labelled a dev fallback in code, in the logs and in the docs - it is not an accuracy claim.	hub/verifier.py
Evidence fusion	Combines the audio score with the node's confidence, motion, darkness and time of day into one severity number, and escalates the mission priority when the evidence is strong.	hub/fusion.py
Packet security	AES-128 encryption plus a signature, with a key derived per node from one master key, and a counter that must always increase so an old packet cannot be replayed.	hub/packets.py and firmware/node/node.ino
Haversine distance	Great-circle distance between two coordinates. Used for the geofence, arrival detection, the ETA and choosing the nearest drone station.	flight_core/mavlink_interface.py
Nearest-station dispatch	Picks the drone station closest to the incident. Four stations cover Nagpur north, south, east and west.	hub/sim_drone.py
Obstacle routing	If the straight path would pass too close to a configured no-fly zone, two detour waypoints are inserted to carry the path around it, and the process repeats for further zones. With no zones configured this is identical to flying direct.	flight_core/obstacle_avoidance.py
Verified mode transition	Every flight-mode command is re-sent as raw MAVLink every 700 ms until the autopilot confirms it. Written because the autopilot was found silently ignoring the library's mode command - the software believed one mode while the aircraft was in another.	flight_core/mission_executor.py

Algorithm	What it does	Where it lives
Failsafe arbitration	A background monitor checks link, battery, GPS, geofence and elapsed time once a second, and resolves conflicts by a fixed priority so a mild warning can never override a critical one.	flight_core/failsafe_handler.py
Mission queue	There is one aircraft, so missions run one at a time, highest priority first, oldest first within a priority. The queue is capped so overload becomes an explicit rejection rather than unbounded growth.	trigger_api/mission_queue.py

The four formulas to memorise

THE ONLY MATHS YOU NEED

```
1. Mel scale - why the audio filters are spaced the way they are.
   Human pitch perception is roughly logarithmic, and this matches it:
        mel(f) = 2595 * log10(1 + f / 700)

2. Evidence fusion - the weights add up to 1.00:
        severity = 0.60 * audio_score        (Stage-2 verification dominates)
                 + 0.15 * stage1_confidence  (the node's own opinion)
                 + 0.10 * motion             (PIR sensor)
                 + 0.08 * darkness           (from the light sensor)
                 + 0.07 * night              (1 if between 20:00 and 06:00)

3. The dispatch decision - two independent gates, both must pass:
        audio_score >= 0.50    AND    severity >= 0.60

4. Haversine distance between two coordinates (R = 6,371,000 m):
        a = sin^2(dlat/2) + cos(lat1) * cos(lat2) * sin^2(dlon/2)
        d = 2 * R * arcsin(sqrt(a))
```

MFCC in six steps (say it like this)

1. Pre-emphasis - a light high-pass filter that lifts the quieter high frequencies.
2. Split into overlapping 32-millisecond frames and taper each one with a Hamming window, so the frame edges do not create false frequencies.
3. FFT each frame to get its power spectrum.
4. Pass that through 40 triangular filters spaced on the mel scale, so more resolution is spent where human hearing actually resolves detail.
5. Take the logarithm, which compresses the dynamic range and makes loudness additive.
6. Apply a discrete cosine transform and keep the 13 lowest coefficients - adjacent filters are highly correlated, and this concentrates the information into a few numbers.

Why AES-128 specifically - the security follow-up

"Why AES-128 and not something else" is really three separate questions. Separate them, because the answers are different.

The question	The answer
Why AES and not another cipher?	DES and 3DES have been brute-forced or formally retired; RC4 is broken; Blowfish and Twofish are not broken but are far less scrutinised. ChaCha20 is a genuinely good alternative and faster in software - but the ESP32 family has a dedicated AES hardware accelerator, so AES is effectively free on the node, where the energy budget matters most. AES is also a 25-year-old public standard, and exists identically in mbedtls on the ESP32 and PyCryptodome on the Pi - which is what let the two implementations be about 40 lines each and provably produce the same bytes.

The question	The answer
Why 128-bit and not 256?	128 bits means 2^128 possible keys - brute force is physically infeasible, so there is no security margin left to gain. AES-256 uses 14 rounds instead of 10, roughly 40% more compute and energy per block, which is a real cost on a solar-powered node for zero benefit. The payload is 8 bytes of sensor readings; nothing about it justifies a larger key. It also falls out of the key derivation cleanly - the signature function gives 32 bytes and we take the first 16. BE READY FOR THE COUNTER-ARGUMENT: a quantum computer running Grover's algorithm would halve the effective strength to 2^64, which is the legitimate case for AES-256. Our threat model is someone with a cheap radio near a pole, and the protected data is a distress flag with a useful life of about one second - but moving to 256 is a one-line change if that ever mattered.
Why a shared key and not RSA or ECC?	Packet size. The whole packet is 25 bytes. An RSA-2048 signature is 256 bytes - ten times the entire packet - and even the compact option, ECDSA on P-256, is 64 bytes, still 2.5 times the packet, on a radio where airtime scales directly with length. The computation cost on the node is far higher too. Structurally, public-key crypto exists to let strangers verify each other without a shared secret, and this is a closed system - we own every node and the hub. THE TRADE-OFF TO ADMIT: compromising the master key compromises the network. Deriving a separate key per node limits lateral damage - stealing one pole gives an attacker nothing for the next one - but the master is still a single point of failure, which is why it lives only on the hub and node keys are burned in at provisioning.

THE POINT THAT SCORES HIGHEST HERE

Encryption is not what protects the drone - the signature is. An attacker does not need to READ your packet to cause harm; they need to FORGE one. What actually stops a spoofed launch is the signature, checked in constant time BEFORE anything is decrypted, plus the counter check that rejects replays. So say it as: 'AES-128 for confidentiality, HMAC-SHA256 for authenticity, and an always-increasing counter for replay resistance - and the authenticity check is the one that matters for safety, so it runs first.' If you say only 'we use AES-128', that is the gap a knowledgeable examiner will push on.

The failsafes - know the action and the reason

Failsafe	Trigger	Action	Why that action
Link loss	No autopilot heartbeat for 10 s	Return home	If the heartbeat is stale, every other reading is frozen data - battery reads fine, GPS reads locked. None of it can be trusted.
Critical battery	10% or below	Land now	There may not be enough energy to reach home. Landing here beats falling out of the sky en route.
Low battery	20% or below	Return home	Still enough margin to fly back.
GPS loss	No fix for 3 consecutive seconds	Land now	Without a position fix, return-to-launch literally cannot navigate. The 3-second debounce means one noisy reading never puts the aircraft down.
Geofence breach	More than 5 km from home	Return home	Operational and regulatory boundary.
Mission timeout	Flight longer than 30 minutes	Return home	Hard upper bound; catches any stuck state the phase logic missed.

6 Design choices, alternatives, and why

The comparison table. If you can give the last column for any row, you can defend the whole design.

Decision	What we use	Alternatives considered	Why this one
Microphone	INMP441 digital I2S MEMS mic	Analog electret with an amplifier; USB mic	The signal is digital from the mic chip onward, so a pole-mounted node with long power runs cannot inject electrical hum into the audio. Needs no analog front end, costs about 200 rupees.
Node processor	ESP32-S3	Classic ESP32; Arduino Nano 33 BLE Sense; a Raspberry Pi at every pole	The S3 has the vector instructions the on-device model needs and can address external RAM for the audio buffer - the classic ESP32 has neither. A Pi per pole would triple the cost and power for no detection benefit.
Where Stage 1 runs	On the node, on-device	Stream all audio to the hub and classify centrally	Streaming needs WiFi or 4G at every pole - cost, power, coverage - and turns every node into a live microphone, which is a privacy problem. On-device means under 0.2% of audio is ever transmitted.
Feature extraction	MFCC, hand-written identically in Python and C	The librosa library; a log-mel spectrogram; Edge Impulse	The single most common cause of a model that works in the lab and fails on the device is a mismatch between training features and device features. One hand-written definition, mirrored step for step, plus a test that checks it, makes that mismatch impossible to miss.
Stage-1 model	26 inputs, 24 hidden, 4 outputs. 748 parameters, about 3 KB	A convolutional network under TensorFlow Lite Micro (this code path exists); SVM; decision tree	It trains with NumPy alone and deploys as three matrix multiplies with NO machine-learning library on the device. The convolutional version is strictly better on accuracy and is the next step - it is written, not yet trained.
Alert radio	LoRa at 433 MHz	4G module; NB-IoT; WiFi mesh; Zigbee	5 to 10 km per hop with no SIM, no subscription and no internet dependency - so it keeps working in exactly the low-infrastructure areas the project targets. 4G would also be a recurring cost per node forever.
Packet contents	25 bytes: node ID, counter, encrypted sensor data, signature	JSON over the radio; putting GPS coordinates in the packet	Radio airtime is the scarce resource. The node sends only its ID and the hub's registry supplies the surveyed coordinates - so no node needs a GPS module at all, saving cost, power and fix-acquisition delay.
Packet security	AES-128 encryption plus a signature plus an always-increasing counter	No encryption; AES-GCM or CCM (single-pass authenticated encryption); LoRaWAN's own security	A spoofed packet would launch an aircraft, so authentication is mandatory. These exact primitives exist on both the ESP32 and the Pi, which made the two implementations short and provably identical. GCM or CCM would be the cleaner next version - be ready to say that.
Stage-2 model	PANNs CNN14, pretrained on AudioSet	YAMNet; an audio transformer; training our own from scratch	AudioSet is 2 million labelled clips - orders of magnitude more supervision than this project could ever collect - and it already contains screaming, shouting and crying classes. Fine-tuning it on local data is the correct next step.
Evidence fusion	A transparent weighted sum of five signals	A trained classifier; logistic regression; Dempster-Shafer; fuzzy rules	Every dispatch must be explainable after the fact, and the code carries the full evidence trace into the log and dashboard. Also, nobody has labelled incident data yet, so a learned fuser would have nothing to learn from. The weights are declared as prototype values, not learned ones.
Decision rule	Two independent thresholds	One threshold; a learned decision boundary; a human dispatcher in the loop	The two gates test different things: 'was that really distress' and 'does this situation warrant an aircraft'. A human in the loop is a legitimate alternative and would raise precision, at the cost of the response time the project exists to cut.
Autopilot	ArduPilot (stock firmware on the Pixhawk)	PX4; writing our own control loops	ArduPilot has the most mature autonomous-mode and failsafe stack and a first-class command interface. PX4 would work with minor changes. Writing our own flight controller would be reckless.

Decision	What we use	Alternatives considered	Why this one
Flight library	dronekit, with a compatibility shim for modern Python	pymavlink directly; MAVSDK; ROS 2	Its high-level vehicle object made the state machine readable quickly. But it is UNMAINTAINED - it needs a shim just to import on Python 3.10+. Migrating away from it is our number-one engineering task. Volunteer this, do not wait to be caught on it.
Command delivery	Confirm every mode change against autopilot telemetry before proceeding	Send and assume it worked (which is the library's default)	We found the autopilot silently ignoring the library's mode command - the software believed the aircraft was in guided mode while it was still in manual. This is the project's core safety contribution and one of two patent drafts.
Abort behaviour	Block until the aircraft has landed AND disarmed	Return as soon as return-to-launch is commanded	Returning early would let the queue start the next mission against an aircraft that is still in the air. The rule is absolute: the queue can never fly against an armed aircraft.
Obstacle avoidance	Geometric routing around operator-configured no-fly zones	Sensor-based reactive avoidance using a rangefinder or depth camera	Pure geometry needs no extra hardware, is fully testable without a simulator, and reduces exactly to flying direct when no zones are configured. Sensor-based avoidance is the honest roadmap item - be clear that we do not have it.
Payload release	A hobby servo commanded over MAVLink	Electromagnet; solenoid; landing to hand the kit over	One standard command works identically in the simulator and on hardware. The design rule is that a failed release never causes the aircraft to loiter - it reports the failure and returns home.
Dashboard	React with a Leaflet map, offline tiles	Mission Planner or QGroundControl; plain HTML	The audience is a police or security operator, not a drone pilot: one map, one incident list, dispatch and recall. It deliberately exposes no flight controls beyond those two. Leaflet works without internet.
Testing	80 fast tests with mocked vehicles, plus one real end-to-end simulated flight	Manual flight testing only; unit tests only	The safety logic is tested against a mock autopilot that accepts a command but never adopts it - which is exactly the real-world failure we found, and which a simulator cannot reproduce on demand. The 6-minute simulated flight then proves the whole chain.
Demo without hardware	A phone browser drives the real detection code end to end	Wait for hardware; show a pre-recorded video	It exercises the REAL Stage 1, Stage 2 and decision code on REAL audio from a phone microphone, so the pipeline is proven before any hardware exists. It explicitly does not test radio range, the ESP32, or physical flight - and we say so.

7 Numbers and libraries they will ask about

The numbers

Value	What it is	Why that number
16,000 Hz	Audio sample rate	Covers up to 8 kHz, and human screams peak between 1 and 4 kHz - so the whole informative band, at a quarter of CD data rate
2 seconds	Classification window	Long enough to contain a whole scream or the word 'bachao'; short enough to run inference twice a second
4 seconds	Audio buffer on the node	Circular, so the uploaded clip includes the audio from before the trigger - the onset is what Stage 2 needs most
13	Feature coefficients kept	Standard for speech: the low coefficients carry the spectral shape, the high ones mostly noise
40	Mel filters	Enough frequency resolution without over-fragmenting the spectrum
123 x 13	Feature matrix per window	Derived from the window length and frame spacing, not chosen
748	Parameters in the deployed model	About 3 KB of numbers - it has to fit on a pole with no ML library
4	Classes	background, scream, cry, help. Background must be class 0 - the firmware tests for 'not background'
0.60	Node trigger confidence	Tuned for recall, not precision. Stage 2 removes the false positives, so a missed event is the worse error here
15 seconds	Refractory period after an alert	One incident should produce one alert, not thirty - protects both the radio channel and the drone queue
433 MHz, SF9	LoRa frequency and spreading factor	Licence-free band in India; lower frequency penetrates buildings better. SF9 balances range against airtime. Node and gateway must match
25 bytes	Alert packet size	9 header + 8 encrypted + 8 signature. Comfortably one radio frame
0.50 / 0.60	The two dispatch thresholds	Audio score, then fused severity. Deliberately on different quantities so they test different things
8 seconds	How long the hub waits for the clip	Enough for a WiFi upload; short enough that a missing clip does not delay a real emergency
15 m	Cruise altitude	High enough to clear people and trees, low enough to keep the incident in camera frame
2 to 120 m	Allowed altitude range	120 m is the small-drone ceiling in most jurisdictions including Indian rules. Enforced when the request arrives, not in flight
5 m	Waypoint arrival tolerance	GPS noise alone is 1 to 3 m, so tighter would risk never registering arrival. The simulated flight measured 0.6 m closest approach
5 km	Geofence radius	A target outside it is rejected with an error on the ground, rather than aborting mid-air
20% / 10%	Low and critical battery	20% leaves margin to fly home; 10% means land now, wherever you are
3 m	Kit drop altitude	Low enough not to damage the kit, high enough to stay clear of people
700 ms	Mode-command retry interval	Faster than the roughly 1 Hz heartbeat, so several attempts land inside one confirmation window without flooding the link
30 minutes	Maximum mission duration	Hard upper bound on any single flight

The libraries that matter

Library	Side	What it does for us	Alternative
dronekit + pymavlink	Python	Talks to the autopilot: mode, arm, takeoff, navigate, read telemetry. pymavlink also sends the raw commands the confirmation loop needs	dronekit is unmaintained; migrating fully to pymavlink is roadmap item one

Library	Side	What it does for us	Alternative
dronekit-sitl	Python	Simulates an ArduPilot aircraft so the whole stack can fly with no hardware	ArduPilot's own newer simulator - the upgrade path
FastAPI + pydantic	Python	The HTTP interface, and bounds checking on every incoming request so the flight code never sees an impossible coordinate or altitude	Flask (no built-in validation, no async)
numpy	Python	All the array maths: the transform, the filters, and the entire model forward and backward pass	Not optional anywhere - it is the only hard dependency of the deployed model
pycryptodome	Python	AES-128 encryption on the hub side (the signature comes from the standard library)	cryptography; the standard library has no AES
panns-inference + torch	Python	The Stage-2 audio model. Optional - about 2 GB, installed on the Pi only, and the code falls back cleanly without it	YAMNet; an audio transformer
tensorflow (optional)	Python	Trains the convolutional Stage-1 model and shrinks it to 8-bit for the microcontroller	Deliberately optional - the NumPy trainer needs none of it
pytest	Python	The 80-test suite	unittest
LoRa (Sandeep Mistry)	C++	Drives the radio module on both the node and the gateway	RadioLib (more capable, more complex)
mbedtls	C	AES and the signature on the ESP32 - the exact mirror of the Python side, and it uses the chip's AES hardware accelerator	Bundled with the ESP32 toolchain, so no extra dependency
Preferences (NVS)	C++	Stores the message counter so it survives reboots - without this, replay protection would break on every power cycle	EEPROM emulation
React + Leaflet	JS	The live operator map: node markers, drone position, flight trail	Mapbox or Google Maps, both of which need an API key and internet

8 What is proven, and what is not

THE MOST IMPORTANT SLIDE DISCIPLINE IN THIS PROJECT

Do not put an accuracy percentage on a slide. The current model was trained on a generated bootstrap dataset whose only purpose is to prove the pipeline runs end to end. Quoting a number from it is the fastest way to lose a viva. Say instead: 'the pipeline is validated end to end; measured detection metrics are the Phase-1 deliverable, and the training script already reports precision, recall, F1, a confusion matrix and the false-alarm rate.'

Proven

- 80 unit tests pass in about 21 seconds - covering failsafe priority logic, the command-confirmation loop, the abort interlock, packet encryption, tamper, wrong-key and replay rejection, the hub decision chain, feature extraction and obstacle geometry.
- End-to-end autonomous flight in the ArduPilot simulator: five consecutive passes, one run logged at 331 seconds with 0.6 m closest approach to the target and all 8 required checks met.
- The full chain with zero hardware: a synthesised scream is sealed into a packet, authenticated by the hub, verified, fused, dispatched, and flown to completion including the kit drop and return.
- The phone demo path: real audio from a phone microphone through the real Stage-1 and Stage-2 code into a real dispatch decision.

Not yet done - name these yourself before the panel does

Gap	How to answer it
No hardware built or flown	Everything is validated in simulation and on the phone path. The hardware plan is four phases: audio bench, radio range, drone build with mandatory manual override and visual line of sight, then the integrated field demo.
The node firmware has never been compiled on hardware	Its own header says so. The first Phase-1 task is confirming the device's feature extraction matches the Python version on a shared test clip, because everything downstream depends on that.
Detection accuracy is unmeasured	The measurement harness is already written and reports honest per-class metrics. It needs real recordings, which is Phase 1.
Stage 2 runs the simple fallback on any dev machine	The real model needs a 2 GB install that only lives on the Pi. The fallback is labelled as such in the code, the logs and the docs.
The flight library is unmaintained	dronekit needs a compatibility shim just to import on modern Python. Migrating to pymavlink is our top engineering task, and it would also unlock the autopilot's own obstacle avoidance.
No sensor-based obstacle avoidance	Only routing around zones we configure on a map. Reactive avoidance needs a rangefinder or depth camera plus newer autopilot firmware.
The kit release is not physically confirmed	We confirm the servo command was sent, not that the kit left the aircraft. A payload switch is named as future work.
The encryption keys in the repository are development keys	A real deployment must provision a per-site master key from the environment, not the default in the source.
Patents not filed	Two draft specifications exist. Worth flagging the risk out loud: the repository is public, which counts as self-disclosure, so filing is time-critical.
No continuous integration, Docker path unverified	Both are known roadmap items. The Docker setup was repaired by review but has never been run - there is no Docker on the development machine.

9 Twenty-five rehearsed questions

Read these aloud once. The wording deliberately volunteers the limitation before the panel has to dig for it - that scores higher than bluffing.

Q. In one sentence, what is your project?

A. A network of listening posts that detects human distress on-device at the street pole, verifies it with a bigger model at a local hub, and automatically sends a drone to the verified location with a camera and a first-aid kit - all without internet.

Q. Why not just a mobile app or a panic button?

A. Both need the victim to act - reach a phone, unlock it, press something. Acoustic detection is passive: a scream is involuntary. They are complementary, and our dispatch interface would accept an app-generated alert unchanged.

Q. What is actually novel? Sending a drone to a GPS point already exists.

A. Agreed, and our own prior-art scan says exactly that - we found three granted patents covering trigger-to-GPS dispatch. What we claim is the verification and safety layer: two independent stages of acoustic verification before an aircraft is ever committed, and a flight stack where every command is confirmed by the autopilot before the mission proceeds.

Q. Which code goes onto which hardware?

A. Two ESP32 boards get code we wrote and burned: the sensing node firmware and the radio gateway firmware. The Pi 5 hub runs our Python hub package. The small computer on the drone runs the flight core and the trigger API. The Pixhawk runs stock ArduPilot - none of our code goes on it; we only send it MAVLink commands.

Q. Why two stages instead of one good model?

A. The good model is about 80 MB of weights. An ESP32-S3 has about 512 KB of memory. No single model fits both. The cascade also buys three things for free: over 99% less audio transmitted, no continuous audio leaving the pole, and a power budget a solar node can actually meet.

Q. Why LoRa and not 4G?

A. No SIM, no subscription, no internet, 5 to 10 km per hop. The places that need this most are exactly the ones with the least reliable connectivity - and 4G would be a recurring cost per node forever.

Q. How does the hub know where the incident is if the packet is only 25 bytes?

A. The packet carries a node ID, not coordinates. Each pole is surveyed once at installation and stored in a registry, so the hub looks up the location. That saves a GPS module per node, its power draw, and its fix-acquisition delay.

Q. What happens if the audio clip never arrives?

A. The system degrades rather than failing. The audio score falls back to 60% of the node's own confidence, so a dispatch is still possible on the radio alert alone - but it now needs a much stronger node confidence to clear the gate. Fail-degraded, not fail-open.

Q. Explain your feature extraction in your own words.

A. It converts 32,000 raw samples into a compact description of the shape of the sound spectrum, the way human hearing perceives it. Six steps: a light high-pass filter, splitting into 32-millisecond windows, a frequency transform, 40 filters spaced logarithmically to match human pitch perception, a logarithm, and a final transform that keeps the 13 most informative numbers.

Q. Why write that by hand instead of using a library?

A. Because the model trains in Python and runs in C on the microcontroller. A library's exact internal normalisation is awkward to reproduce in C, and even a small mismatch silently degrades a deployed model. One hand-written definition, mirrored step for step, with a test that checks it, makes that mismatch impossible to miss.

Q. Why is your on-device model so small?

A. Because it lives on a pole. It is 748 parameters, about 3 kilobytes, and needs no machine-learning library at all - three matrix multiplications. That is deliberate: Stage 1 only decides 'worth a look'. The hub does the hard classification.

Q. What is the model you use at the hub, and why not train your own?

A. PANNs - a network pretrained on Google's AudioSet: two million clips across 527 sound classes, already including screaming, shouting, yelling and crying. That is orders of magnitude more supervision than this project could collect. Fine-tuning it on local data is the correct next step and would beat both training from scratch and using it off the shelf.

Q. Where did your fusion weights come from?

A. They are prototype values, and the code says so explicitly. They sum to 1.00 and encode a deliberate ordering: the verified audio score dominates, the node's own confidence adds a little, then motion, darkness and night hours. Tuning them against bench data is the plan. Claiming they are learned would be false.

Q. Why not learn the fusion instead?

A. Two reasons. There is no labelled incident data to learn from yet. And every dispatch has to be explainable afterwards - the code carries the full evidence trace into the log and the dashboard. A learned model trades that away.

Q. Where is your shortest-path algorithm? I do not see Dijkstra or A-star.

A. A quadcopter is not on a road graph, so the straight line IS the true path - which is why choosing the nearest station is simply a minimum over great-circle distances. Graph search belongs where there are obstacles, and there we use deterministic geometry to route around configured zones. If those zones ever got dense, a visibility graph with A-star would replace it.

Q. Why great-circle distance and not simple Euclidean?

A. A degree of longitude shrinks as you move away from the equator. At Nagpur's latitude, treating degrees as a flat plane would misjudge distances by around 7% - enough to put a 5 km geofence in the wrong place.

Q. How do you stop someone spoofing an alert and launching your drone?

A. Every packet carries a signature computed with a key unique to that node, derived from a master key. The hub verifies that signature - in constant time, so there is no timing leak - BEFORE it decrypts anything, and rejects any message counter that is not higher than the last one seen. Tamper, wrong-key and replay rejection are all covered by unit tests.

Q. Would you change anything about the security design?

A. Yes - a single-pass authenticated encryption mode, instead of composing encryption and a signature by hand. It is what the LoRa standard itself uses. Also, the key in the repository is a development key; a real deployment must provision one per site.

Q. What is the single most important safety mechanism?

A. Command confirmation. Every flight-mode change is a request, not a fact: it is re-sent every 700 milliseconds until the autopilot's own heartbeat reports the new mode back. It exists because we caught the autopilot silently ignoring the library's command - the software believed the aircraft was in guided mode while it was still in manual.

Q. What if two failsafes fire at once?

A. There is an explicit priority rule. Landing outranks returning home and is never downgraded, so a 19% low-battery warning can never override a 9% critical one. And the return loop re-checks that priority every second, so it will switch to landing mid-return if the situation worsens.

Q. Why does GPS loss cause a landing rather than a return home?

A. Because with no position fix, returning home cannot navigate - it has no idea which way home is. Landing in place is the only safe autonomous response. And it is debounced over three consecutive seconds, so one noisy reading never puts the aircraft down.

Q. You disabled the pre-arm safety checks. Isn't that dangerous?

A. Only in the simulator, and only when a specific environment variable is set. The simulated autopilot refuses to arm without a radio transmitter, which a simulator does not have. On real hardware that variable is unset and the code logs that it is leaving all checks at stock values without touching a single parameter. Separately, every real flight requires a transmitter with a mode switch as the human kill path.

Q. Do you have obstacle avoidance?

A. Map-based, yes - deterministic routing around no-fly zones we configure, with dedicated tests. Sensor-based reactive avoidance, no. Nothing in the code reads a rangefinder. That needs extra hardware plus newer autopilot firmware, and it is on the roadmap.

Q. What is your detection accuracy?

A. I will not quote one, because the current model was trained on a generated dataset that exists to validate the pipeline, not to measure detection - quoting from it would be misleading. The measurement harness is already written and reports per-class precision, recall, F1, a confusion matrix and the false-alarm rate, and it splits the data by file so augmented copies cannot leak across the split. Producing those numbers on real recordings is the Phase-1 deliverable.

Q. What about privacy? You are putting microphones on public poles.

A. That is precisely why Stage 1 runs on the node. Nothing is streamed - audio sits in a 4-second buffer in memory and is continuously overwritten. Only when a detection fires does a single 4-second clip leave the pole, and it goes to a local hub, not the cloud. So under 0.2% of audio is ever transmitted, and none of it leaves the site.

10 Demo plan and closing notes

If you get to demo, ranked by risk

Demo	Command	Time	Risk
The test suite	python -m pytest	~21 s	Lowest. Run this one. 80 tests green in front of the panel is a strong slide.
Phone demo, no hardware	python -m hub.main --web-only --https, then open the /node page on a phone	~1 min	Low, but both devices must be on the same WiFi and you must accept the self-signed certificate.
Full chain in the simulator	python scripts/demo_phase0.py	~6 min	Medium. Start it before you begin speaking, not while the panel watches.
Live dashboard	.\run_nagpur.ps1, then open localhost:5173	~2 min to boot	Medium - three windows, and the dashboard needs its packages installed.

TEST YOUR DEMO TONIGHT, NOT TOMORROW

This working copy has no virtual environment set up - the project notes point at a different drive. Whichever demo you plan to run, run it once tonight so you find any missing package now rather than in front of the panel.

Your first ninety seconds

1. The problem: distress is detected late, and the gap between 'something happened' and 'someone arrived' is where the harm occurs.
2. The architecture in one breath: tiny model on the pole, big model at the hub, encrypted radio in between, autonomous drone at the end - and nothing depends on the internet.
3. The honest status: the full chain is validated in simulation with 80 passing tests and a repeatable autonomous flight. Hardware bring-up and measured detection metrics are Phase 1.

Three things never to say

- Any accuracy percentage from the current bootstrap dataset.
- That the drone avoids obstacles using sensors. It routes around a configured map.
- That the system has flown on real hardware. It has flown repeatably in the ArduPilot simulator - which is a real and defensible claim on its own.

If the panel wants to read further

Document	What it covers
docs/SEMINAR_VIVA_GUIDE.pdf	The full 40-page version of this brief: every algorithm with its formula and file location, a 31-row decision table, all 60 files, 50 questions
docs/PROJECT_PLAN.md	The master plan: concept, architecture, the four hardware phases, bill of materials, safety, privacy, legal
docs/HARDWARE_INTEGRATION.md	Every pinout, autopilot parameter, calibration step, and the simulator-to-hardware switchover
docs/DATASET_AND_TRAINING.md	Classes, data sources, augmentation, and the metrics to report
docs/RESEARCH_PAPER.md, THESIS.md, patents/	The pre-print, the nine-chapter thesis, two patent drafts; plus SYSTEM_DOCUMENTATION.md for the flight-stack operator reference

LAST THING

The strongest position in a viva is not 'everything works'. It is 'here is exactly what is proven, here is exactly what is not, and here is the test that will settle it'. This project genuinely is in that position. Lead with the proof, name the gaps yourself, and the panel has nothing left to catch you on.